PID Control Explained

Theory, simulation, real-world problems and tuning

Vichruth | Modern Monk Labs Technical Assignment | Code:
`github.com/vichruth/ModernMonks-PID-Control`

What this document does. It explains PID control from scratch and backs every claim with a simulation written in plain NumPy and Matplotlib (no control library). The test machine is a 1 kg block on a spring and damper that must move to 1 m and stay there. Every figure and number comes from the code in the repository; `python run_all.py` regenerates all of them.

How I used AI. I used AI tools (Gemini and Claude) as pair-programming help for code scaffolding, plotting and proofreading, and for an independent review: a second AI re-ran key results with its own simulation and caught several wrong numbers, which are corrected here. Results that matter (e.g. the steady-state error formula, damping ratio, Ziegler–Nichols K_u and T_u) were also checked against closed-form theory.

1 The basic idea

Think of adjusting a shower. The water is too cold, so you open the hot tap a lot. As it warms up you open it less. If it gets too hot you close it again. You never "know" the right tap position; you keep measuring the gap between what you want and what you have, and you act on that gap. This is *feedback control*.

In engineering words: the **setpoint** r is the target, the **plant** is the machine, a **sensor** measures its output y, and the **error** is

$$e(t) = r(t) - y(t). \tag{1}$$

The **controller** turns the error into a command u (here a force in newtons) for the **actuator**. Without feedback ("open loop") the machine just runs a fixed command and cannot react to anything unexpected.

Figure 1. Closed-Loop Negative Feedback Control Architecture

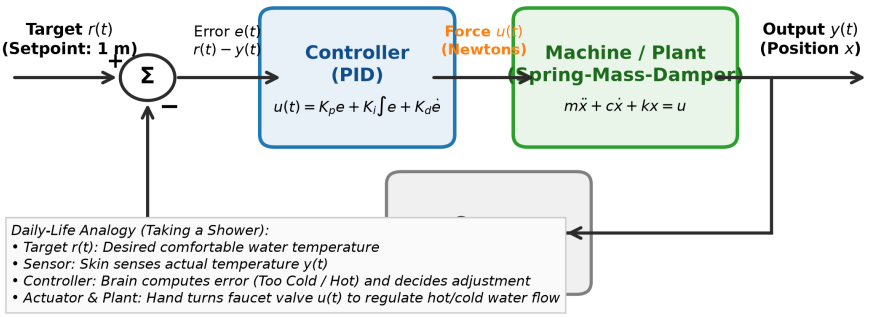


Figure 1: The control loop: target → error → controller → machine → sensor → back to the start. The shower analogy is mapped onto each box.

Worked number. Stepping into the shower at $y = 20°\text{C}$ with a target of $40°\text{C}$ gives $e = +20°\text{C}$, so a big correction. At $39.5°\text{C}$ the error is $+0.5°\text{C}$ and almost no correction is needed.

Takeaway. A controller keeps shrinking the gap between target and reality by acting on the measured error.

2 The PID formula

A PID controller adds up three reactions to the error:

$$u(t) = \underbrace{K_p\, e(t)}_{\text{P: present error}} + \underbrace{K_i \int_0^t e(\tau)\, d\tau}_{\text{I: accumulated past error}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{D: current rate of change}} \tag{2}$$

Symbol	Name	Unit	Meaning
r, y, e	setpoint, output, error	m	target position, measured position, $r - y$
u	control force	N	what the actuator pushes with
K_p	proportional gain	N/m	like a spring: push proportional to how far away you are
K_i	integral gain	N/(m·s)	memory: keeps growing the push while any error remains
K_d	derivative gain	N·s/m	like a damper: pushes against fast motion, braking the approach

Plain-words meaning. P reacts to *how big* the error is now. I reacts to *how long* it has been there. D reacts to *how fast* the error is changing, so it brakes when the target is approached quickly. D does not see the future: it reacts to the current slope, which hints at where the error is heading.

What a computer actually does. It samples every Δt (1 ms here) and repeats:

$$e_k = r_k - y_k \qquad I_k = I_{k-1} + K_i\, e_k\, \Delta t \tag{3}$$

$$P_k = K_p\, e_k \qquad D_k = K_d\, \frac{e_k - e_{k-1}}{\Delta t}, \qquad u_k = P_k + I_k + D_k \tag{4}$$

At $t = 0$ the setpoint steps from 0 to 1 m, so $P(0) = K_p \cdot 1\,\mathrm{N}$; the integral starts at 0 and grows at $K_i \cdot 1\,\mathrm{N/s}$; D appears only once the block moves.

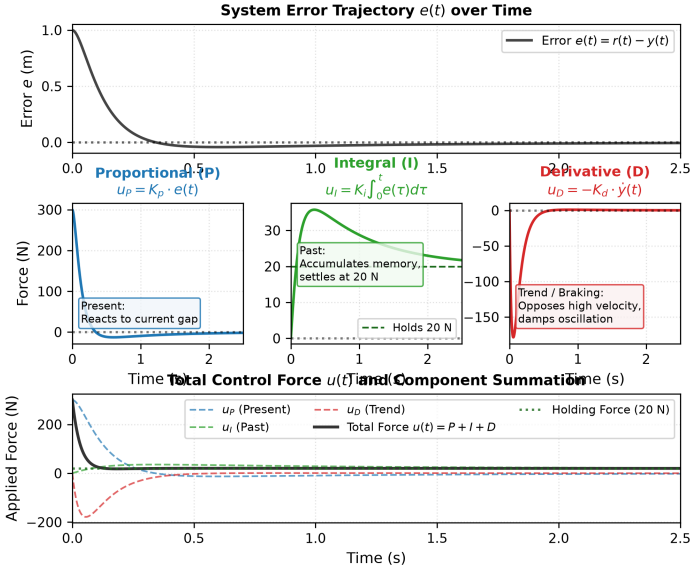


Figure 2: Top: the error shrinking over time. Middle: the separate P, I and D forces. Bottom: their sum, the force actually applied.

Takeaway. P acts on the present, I on the past, D on the trend. The sum of the three is the force.

3 The test system (given)

A block of mass $m = 1\,\mathrm{kg}$ sits on a spring ($k = 20\,\mathrm{N/m}$) with a damper ($c = 10\,\mathrm{N{\cdot}s/m}$). With force u:

$$m\ddot{x} + c\dot{x} + kx = u. \tag{5}$$

The open-loop damping ratio is $\zeta = c/(2\sqrt{mk}) = 10/(2\sqrt{20}) = 1.118 > 1$, so on its own it creeps to its final position without bouncing. Poles: $s = -2.76$ and -7.24.

Why a constant push is needed. Once the block rests at $x = 1\,\mathrm{m}$, velocity and acceleration are zero, so $0 + 0 + k\,x = u$ and

$$u_{ss} = k\, x_{ss} = 20\ \mathrm{N/m} \times 1\ \mathrm{m} = \mathbf{20\ N}. \tag{6}$$

The spring keeps pulling the block back, so something must keep pushing with 20 N. With no push the block returns to 0; with 10 N it settles at 0.5 m (Figure 3).

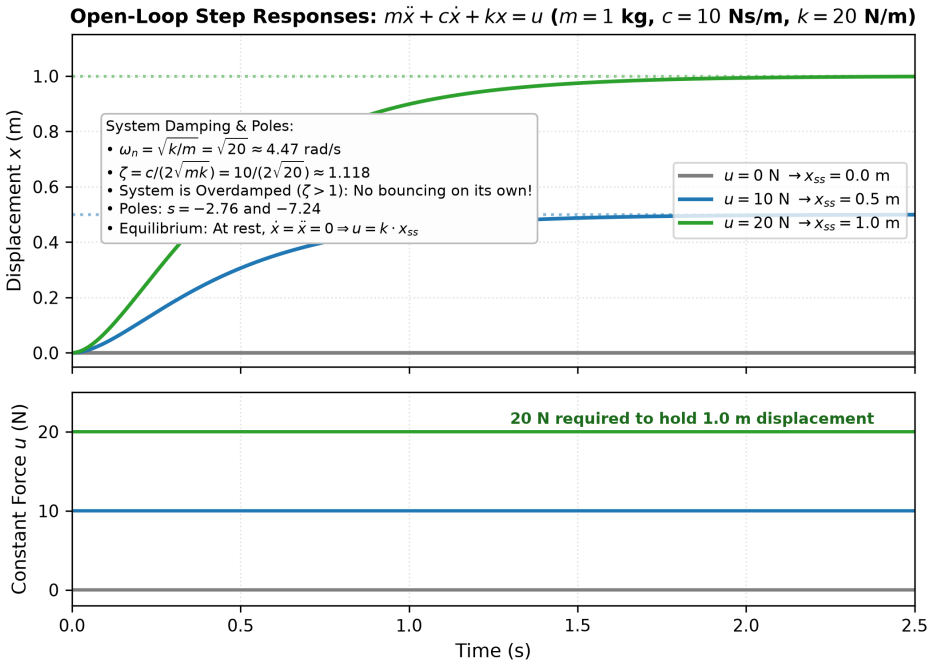


Figure 3: Open-loop responses to constant forces of 0, 10 and 20 N. Only 20 N holds the block at 1 m.

4 How to judge a result

Four numbers describe a step response (all measured on the target $r = 1$ m):

1. **Rise time** t_r: time to go from 10% to 90% of the target. Reported as "n/a" if the response never reaches 90%.
2. **Overshoot**: $\text{OS} = \dfrac{y_{\text{peak}} - r}{r} \times 100\%$, how far past the target it goes (always measured against the setpoint).
3. **Settling time** t_s: the time after which y stays within $\pm 2\%$ of the target (0.98 to 1.02 m). Reported as "never" if it does not.
4. **Steady-state error** e_{ss}: the remaining gap after a long run ($T = 15$ s, averaged over the last 5% of samples).

Figure 4. Step Response Specification Criteria (Rise, Overshoot, Settling, Steady Error)

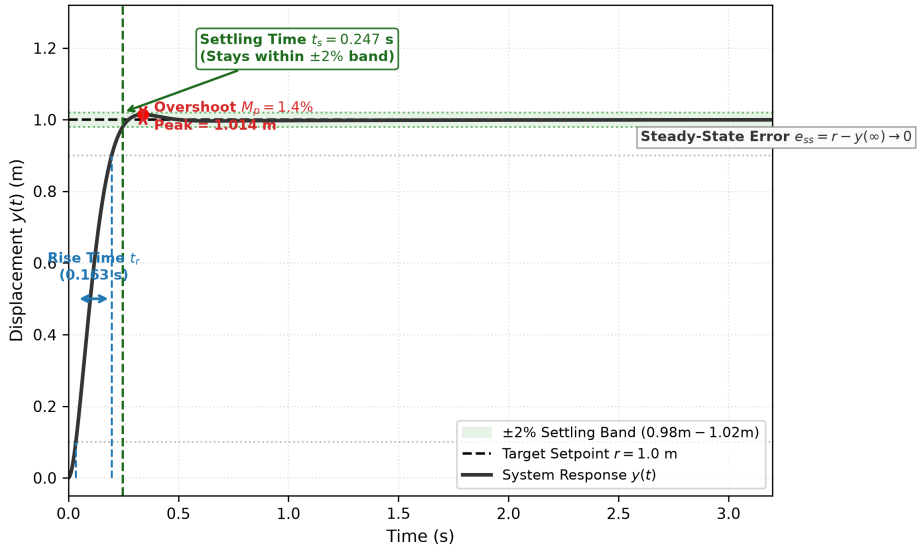


Figure 4: All four measures marked on one example response (PID with $K_p = 220$, $K_i = 180$, $K_d = 14$, chosen only to illustrate the definitions): rise 0.163 s, overshoot 1.4%, settling 0.247 s, steady error 0.0004 m.

Takeaway. A good controller is fast, barely overshoots, settles quickly and ends exactly on target, without demanding an impossible force.

5 Each part alone

5.1 P only: it always stops short

With $u = K_p(r - x)$ the closed loop is $m\ddot{x} + c\dot{x} + (k + K_p)x = K_p r$. At rest this gives $(k + K_p)x_{ss} = K_p r$, so

$$x_{ss} = \frac{K_p}{k + K_p}\, r, \qquad e_{ss} = \frac{k}{k + K_p}\, r. \tag{7}$$

In words: P pushes only while there is an error, and the spring needs a push to hold the block. The block stops where the P-push $K_p\, e_{ss}$ equals the spring force $k\, x_{ss}$; that is always short of the target. For $K_p = 10$ the error is $20/30 = 0.667\,\text{m}$. A bigger K_p shrinks the gap but never closes it, and it lowers the damping ratio $\zeta = c/(2\sqrt{m(k + K_p)})$, so the block starts to bounce.

Figure 5. Proportional-Only Control: Effect of Gain K_p on Steady Error and Oscillation

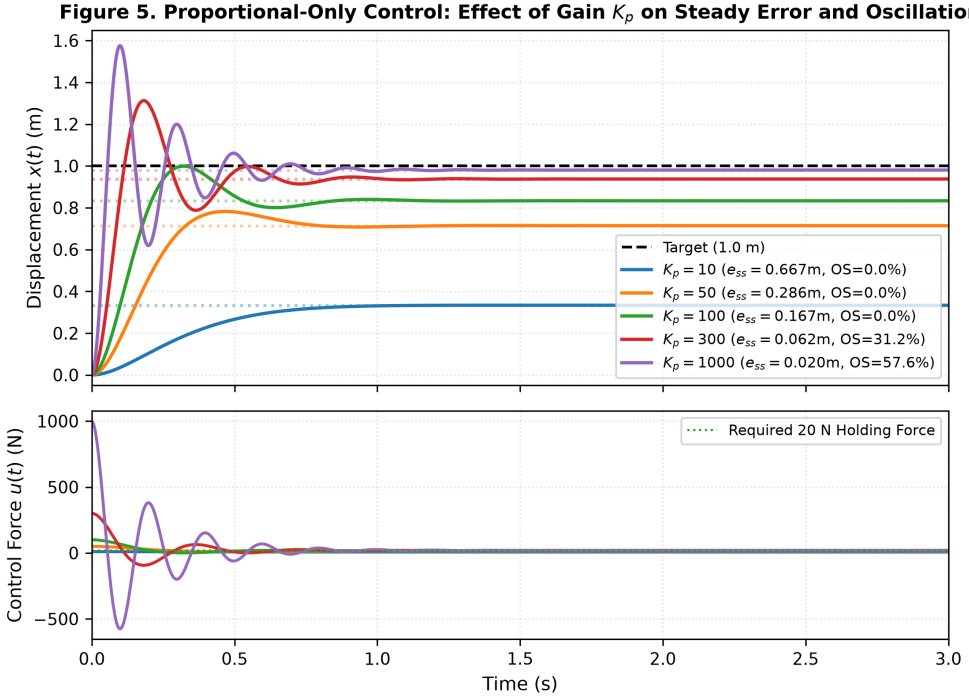


Figure 5: P-only step responses for $K_p = 10, 50, 100, 300, 1000$. Higher gain shrinks the offset (dotted lines are the predicted final values) but adds bouncing and demands a large force.

Figure 6. Theory vs Simulation: Steady-State Offset & Closed-Loop Damping Loss

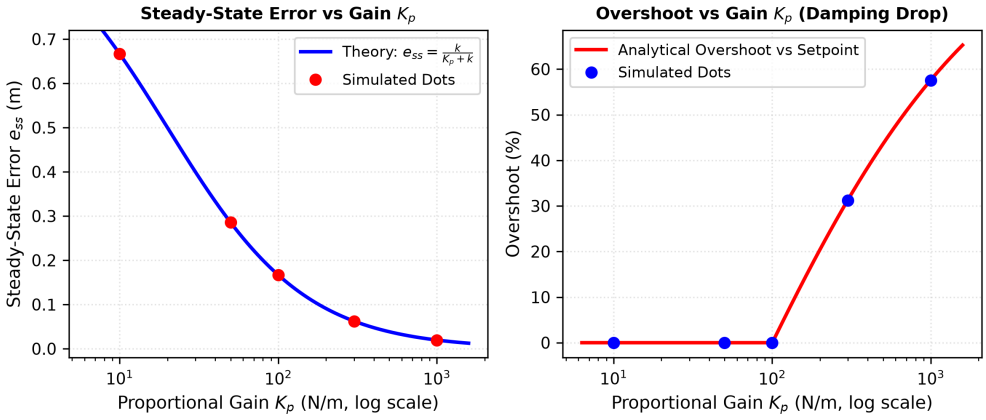


Figure 6: Theory (line) against simulation (dots). Left: steady error $k/(k + K_p)$. Right: overshoot against gain.

K_p (N/m)	ζ	theory e_{ss} (m)	simulated e_{ss} (m)	overshoot	rise time	settling
10	0.913	0.6667	0.6667	0%	n/a	never
50	0.598	0.2857	0.2857	0%	n/a	never
100	0.456	0.1667	0.1667	0%	0.184 s	never
300	0.280	0.0625	0.0625	31.2%	0.077 s	never
1000	0.157	0.0196	0.0196	57.6%	0.036 s	1.43 s

Theory and simulation agree to six decimals. Settling is "never" whenever the offset itself exceeds 2%.

Takeaway. P alone cannot remove the error: $e_{ss} = k/(k + K_p)$. Raising K_p trades a smaller offset for more bounce (57.6% overshoot at $K_p = 1000$).

5.2 Adding I: memory removes the offset

The integral keeps growing while any error remains, so the output can only stop changing when $e = 0$. At that point P and D are both 0, so *the integral term alone supplies the 20 N holding force*: $u_I(\infty) = k\,x_{ss} = 20\,\text{N}$. That is exactly what P alone could not provide.

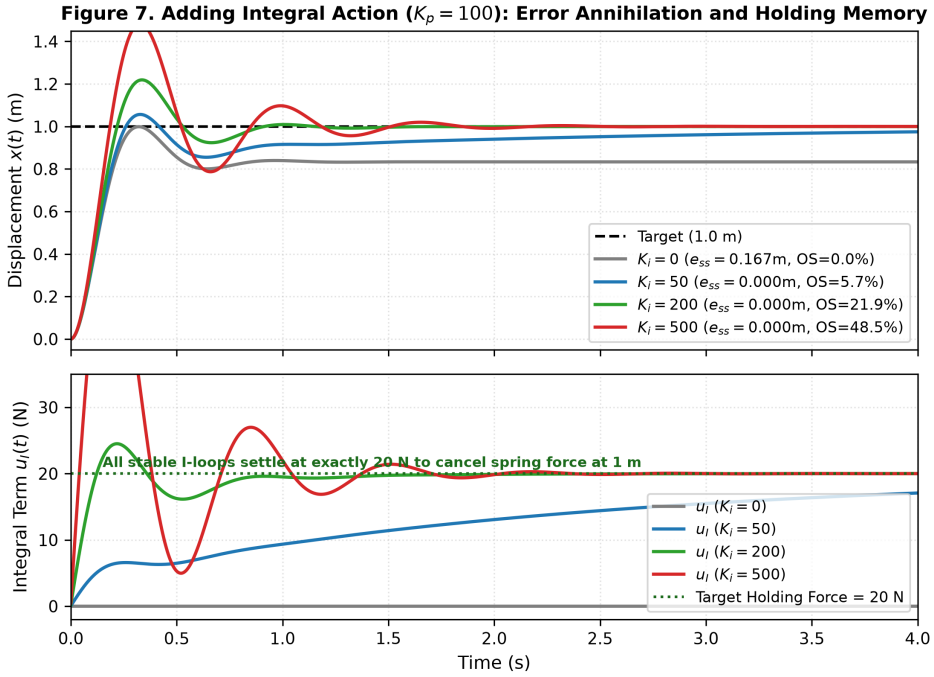


Figure 7: Adding I at $K_p = 100$ ($K_i = 0$ to 500). Steady error goes to zero for every $K_i > 0$ and the integral force settles at 20 N. Larger K_i also brings more overshoot.

5.3 Adding D: damping removes the bounce

Taking D from the measurement ($u_D = -K_d\dot{x}$) simply adds damping, giving $\zeta = \dfrac{c + K_d}{2\sqrt{m(k + K_p)}}$. For $K_p = 300$, $\zeta = 1$ needs $c + K_d = 2\sqrt{320} = 35.78$, i.e. $K_d \geq 25.78\,\text{N·s/m}$. Textbook overshoot $\exp(-\pi\zeta/\sqrt{1-\zeta^2})$ is measured against the *final* value $K_p/(k + K_p) = 0.9375\,\text{m}$; against the setpoint it is $\left[0.9375\,(1 + \text{OS}_{\text{final}}) - 1\right] \times 100\%$.

Figure 8. Adding Derivative Action ($K_p = 300$): Dynamic Braking & Oscillation Quenching

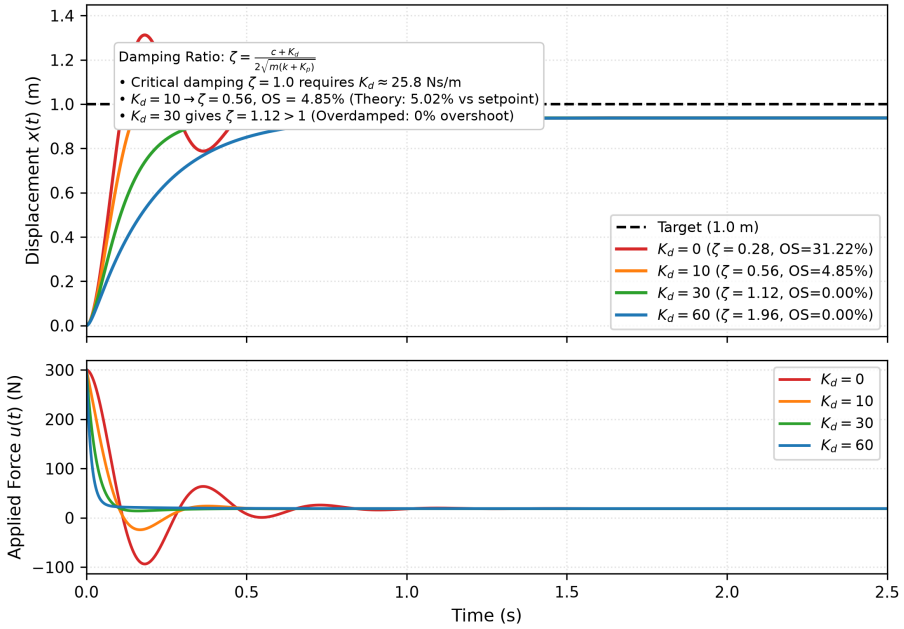


Figure 8: Adding D at $K_p = 300$ ($K_d = 0$ to 60). Overshoot falls from 31.2% to 0% once $\zeta \geq 1$.

K_d (N·s/m)	ζ	OS vs final (theory)	OS vs setpoint (theory)	OS vs setpoint (simulated)
0	0.280	40.07%	31.32%	31.22%
10	0.559	12.03%	5.02%	4.85%
30	1.118	0%	0%	0%
60	1.957	0%	0%	0%

D alone still leaves the 6.25 cm offset (it has no memory), which is why the three terms are combined.

Takeaway. I removes the steady error by learning the 20 N holding force; D adds damping and removes the bounce.

6 All together

Figure 9. Step Response Comparison Across Controller Architectures (P vs PI vs PD vs PID)

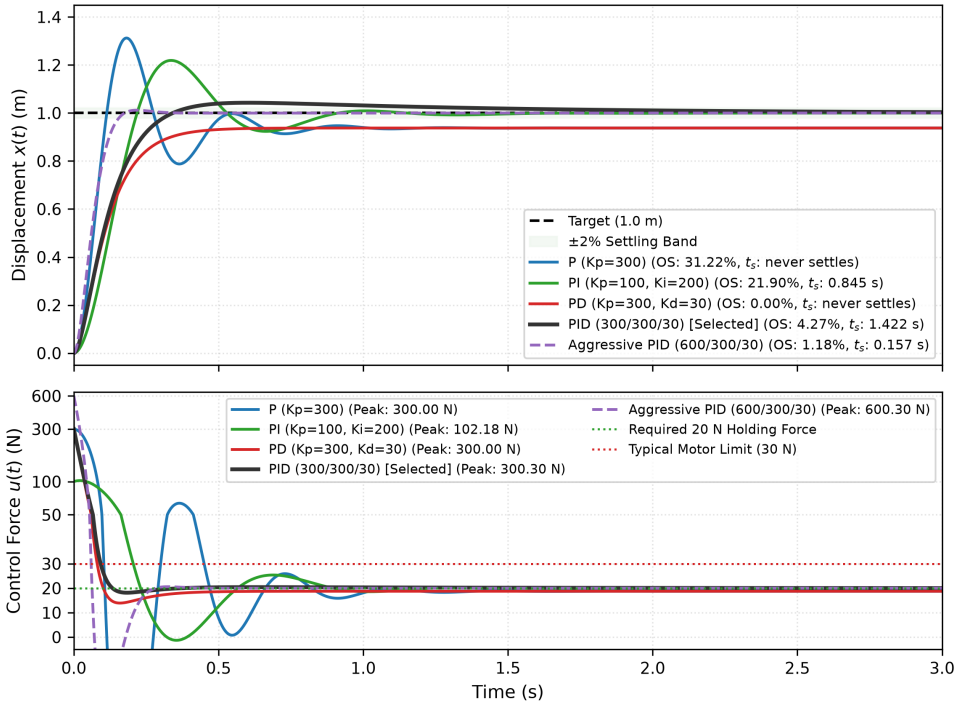


Figure 9: P, PI, PD and PID on one plot (top) with the force each demands (bottom).

Controller	$K_p/K_i/K_d$	rise (s)	overshoot	settling (s)	e_{ss} (m)	peak force (N)
P	300/0/0	0.077	31.22%	never	0.0625	300.0
PI (as in Fig. 9)	100/200/0	0.148	21.90%	0.845	0	102.2
PD	300/0/30	0.305	0%	never	0.0625	300.0
PID	**300/300/30**	0.206	**4.27%**	**1.422**	**0**	300.3
PID, aggressive	600/300/30	0.102	1.18%	0.157	0	600.3

(For reference, PI at 300/300/0 overshoots 38.7% and settles in 0.83 s.)

Only the full PID has no steady error *and* no large overshoot: P gives speed, I accuracy, D calm.

Why $K_p = 300$, $K_i = 300$, $K_d = 30$. These gains meet all of my targets: overshoot below 5% (4.27%), settling below 1.5 s (1.42 s), zero final error. $K_d = 30$ is the round number just above the critical-damping value 25.8 (so $\zeta = 1.12$), and $K_p = 300$ keeps the starting force near 300 N. I also ran a grid search over $K_p \in [200, 600]$, $K_i \in [100, 500]$, $K_d \in [10, 50]$. It shows that faster settings exist: for example 300/200/20 gives 0.46% overshoot and 0.243 s settling at the same 300 N, and 400/200/20 settles in 0.171 s at 400 N. So 300/300/30 is a good, simple baseline, not the optimum. $K_p = 600$ shows the other side of the trade-off: it settles in 0.157 s but demands 600 N at $t = 0$, far above a real 30 N motor, which leads straight into Section 7.

Figure 10. Selected Final Controller: Step Trajectory and Term Contributions

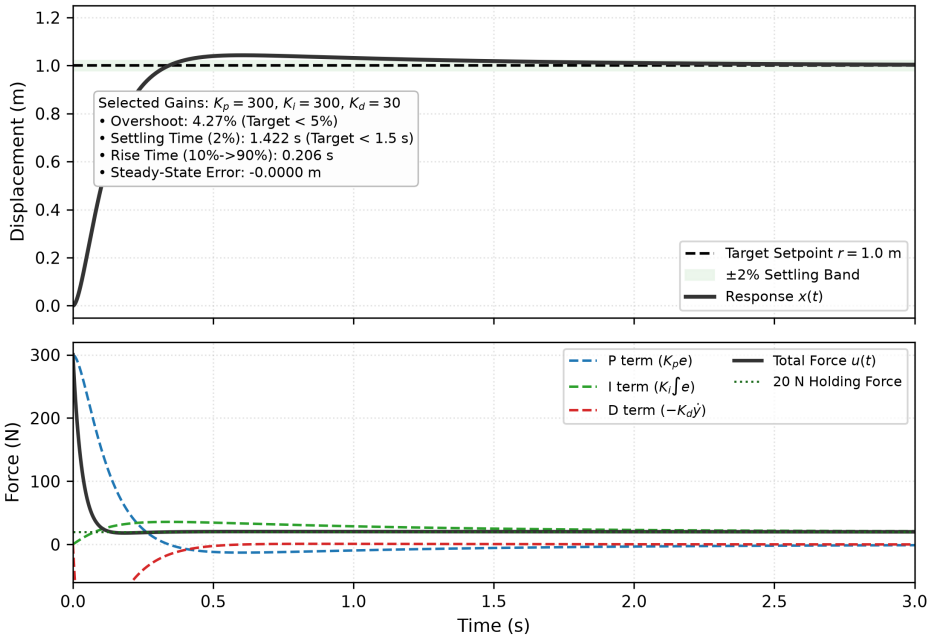


Figure 10: Selected controller: response and force (top); individual P, I and D forces (bottom).

7 Real-world problems

Ideal simulations assume unlimited force, perfect sensors and instant updates. Four things break that, each shown with a graph and fixed. All use gains 300/300/30.

7.1 (a) Actuator limit and integral windup

A real motor saturates (here $|u| \leq 30$ N). While it is saturated the block cannot speed up, but the error is still positive, so the integral keeps filling up. When the block reaches the target the stored charge keeps pushing and it overshoots. **Fix: clamping anti-windup**: stop integrating while the output is saturated *and* the error would push it further into the limit.

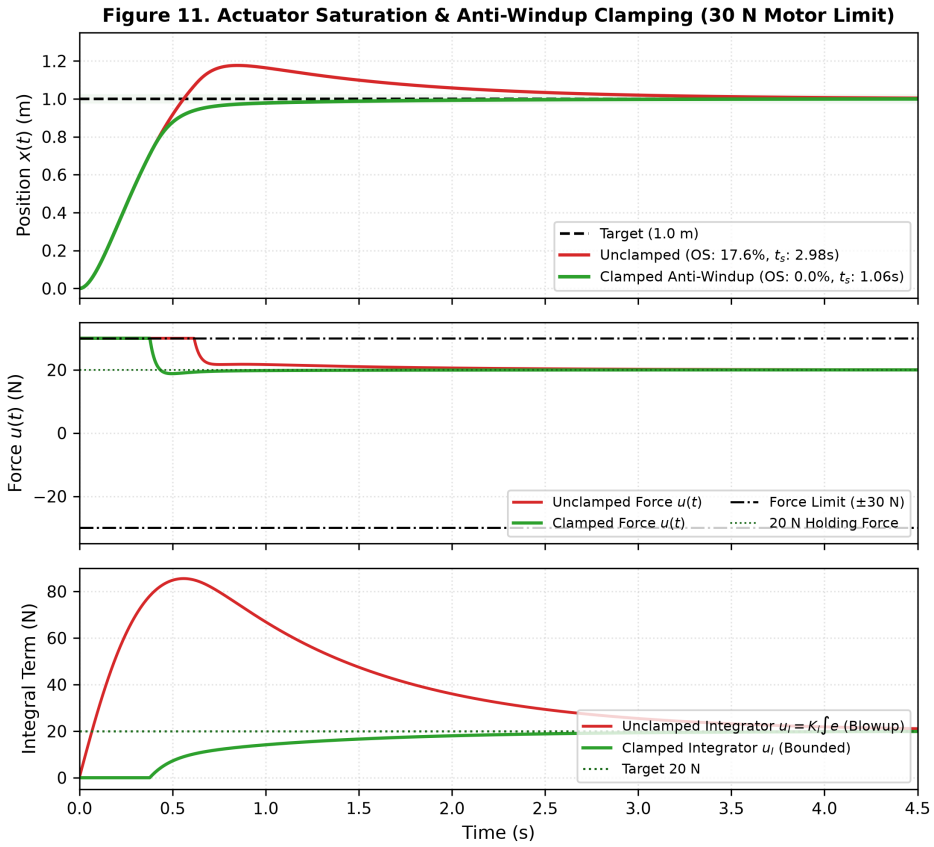


Figure 11: Windup under a 30 N limit. Without the fix: 17.6% overshoot, 2.98 s settling. With clamping: 0% overshoot, 1.06 s.

7.2 (b) Sensor noise and the D term

Real sensors jitter. With Gaussian noise of $\sigma = 1$ mm (seed 42) and $\Delta t = 1$ ms, differencing two samples gives velocity noise of about $\sqrt{2}\sigma/\Delta t = 1.41$ m/s, which $K_d = 30$ turns into

$$\mathrm{std}(u_{\mathrm{noise}}) \approx K_d \frac{\sqrt{2}\,\sigma}{\Delta t} = 30 \times 1.414 \approx 42.4 \text{ N} \tag{8}$$

(measured: 43.4 N), larger than the whole 30 N limit. **Fix: low-pass filter the derivative** with $\alpha = \Delta t/(\tau+\Delta t)$: $\tau = 0.02$ s gives 1.76 N and $\tau = 0.05$ s gives 0.90 N. The price is a small extra lag.

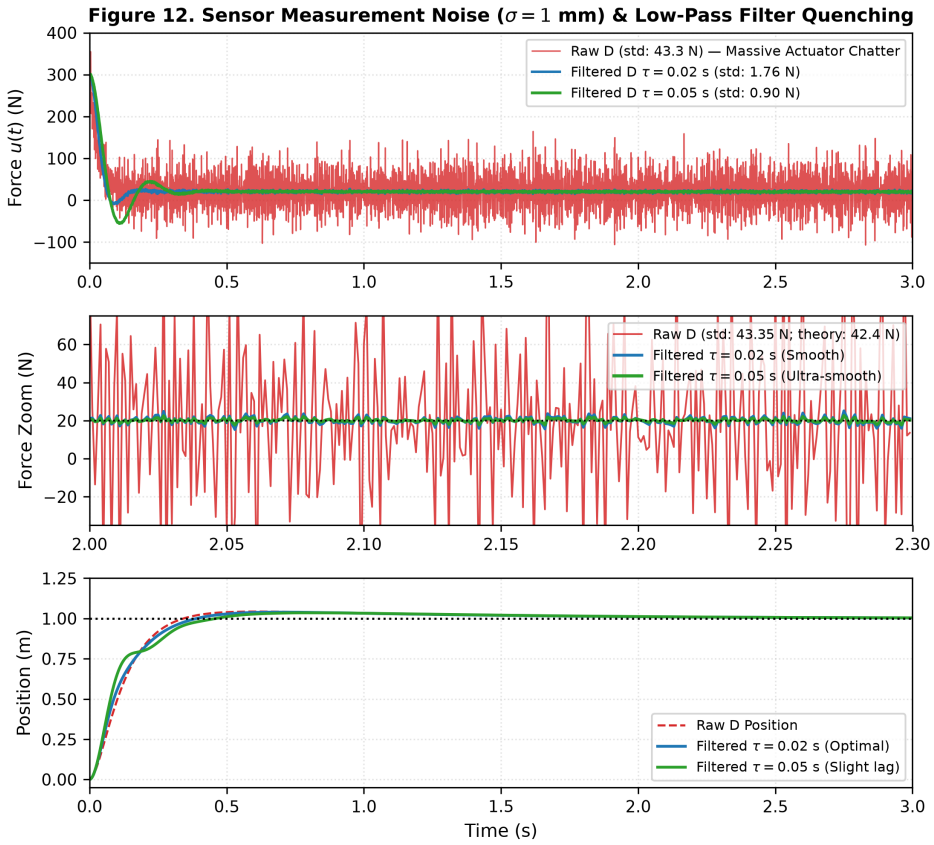


Figure 12: Control force with raw D (large chatter) and with the filtered D ($\tau = 0.02\,\text{s}$).

7.3 (c) Setpoint kick

If D is taken on the error and the target jumps 0 to 1 m in one sample, $de/dt = 1/0.001 = 1000\,\text{m/s}$ and $u_D = 30 \times 1000 = 30{,}000\,\text{N}$, so $u(0) = 30{,}300\,\text{N}$. **Fix: differentiate the measurement, not the error.** The block cannot jump, so the spike disappears (300 N, a 100× reduction).

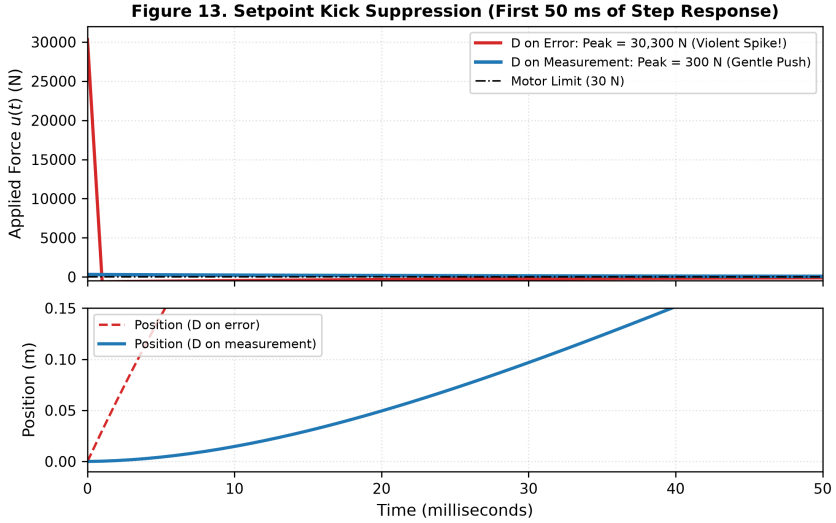


Figure 13: Setpoint kick: D on error gives a 30,300 N spike; D on measurement gives 300 N.

7.4 (d) Slow controller updates

Real controllers update every Δt_{ctrl} and hold the command in between (zero-order hold). That adds a delay of about $\Delta t_{\text{ctrl}}/2$ (a phase lag $\approx \omega\,\Delta t_{\text{ctrl}}/2$), so the controller reacts to old information. I stepped the plant at 1 ms and varied the controller period. Long runs show the stability limit:

- below about 60 ms the response settles (error shrinks over a 60 s run);
- around 62 ms the oscillation grows slowly;
- from about 64 ms upward it grows explosively, and at 90 ms the peak already reaches 160 m within 3 s.

A short 3 s test would report a higher, misleading limit (66.5 ms), because slow growth is invisible over a few

seconds. The system is linear (no saturation) here, so this is steadily growing oscillation, not a limit cycle.

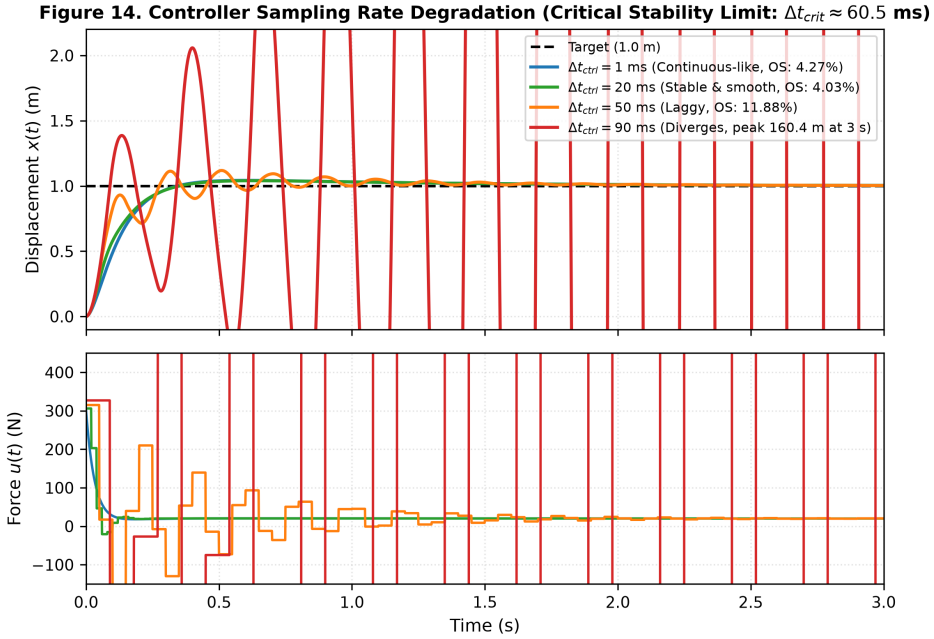


Figure 14: Controller update periods of 1, 20, 50 and 90 ms. Small periods behave like a continuous controller; 90 ms diverges.

Problem	Without fix	Fix	With fix
(a) saturation	17.6% overshoot, 2.98 s	clamping anti-windup	0% overshoot, 1.06 s
(b) noise	43.4 N rms force chatter	low-pass D, $\tau = 0.02$ s	1.76 N rms
(c) setpoint kick	30,300 N spike	D on measurement	300 N
(d) slow updates	diverges at ≥ 64 ms	update faster than about 60 ms	stable, smooth

8 Choosing the gains

8.1 Manual tuning recipe

1. Set $K_i = K_d = 0$.
2. Raise K_p until the response is fast but starts to overshoot ($K_p = 300$: 31.2% overshoot).
3. Add K_d until the bounce is damped ($K_d = 30$: $\zeta = 1.12$, overshoot 0%).
4. Add K_i until the steady error is gone ($K_i = 300$: error 0.0625 m $\to$ 0).
5. If overshoot returns (4.27% here), lower K_i or raise K_d.
6. Re-check with the force limit and with noise (clamping, low-pass on D).

Raise	rise time	overshoot	settling	steady error	noise	example from this report
K_p	falls	rises	varies	falls	little	K_p 100$\to$1000: e_{ss} 0.167$\to$0.020 m, OS 0$\to$57.6%
K_i	small fall	rises	improves, then worsens	removed	none	PD$\to$PID: e_{ss} 0.0625 m $\to$ 0
K_d	rises	falls	improves up to $\zeta = 1$	none	much worse	K_d 0$\to$30: OS 31.2%$\to$0%

8.2 Ziegler–Nichols on $G(s) = 1/(s+1)^3$

Method: use P only and raise K_p until the output oscillates steadily; call that gain K_u and the oscillation period T_u.

Theory check. Three lags have phase $-3\arctan\omega = -180°$ at $\omega = \sqrt{3}$ rad/s, where $|G| = (1/2)^3 = 1/8$. So $K_u = 8$ and $T_u = 2\pi/\sqrt{3} = 3.628$ s.

Simulation. Sweeping K_p from 5 to 10 in steps of 0.1: below 8 the oscillation decays, at 8.0 it is sustained, above 8 it grows. Peaks are 3.625 s apart, so $K_u = 8.00$ and $T_u = 3.625$ s (theory 3.628 s).

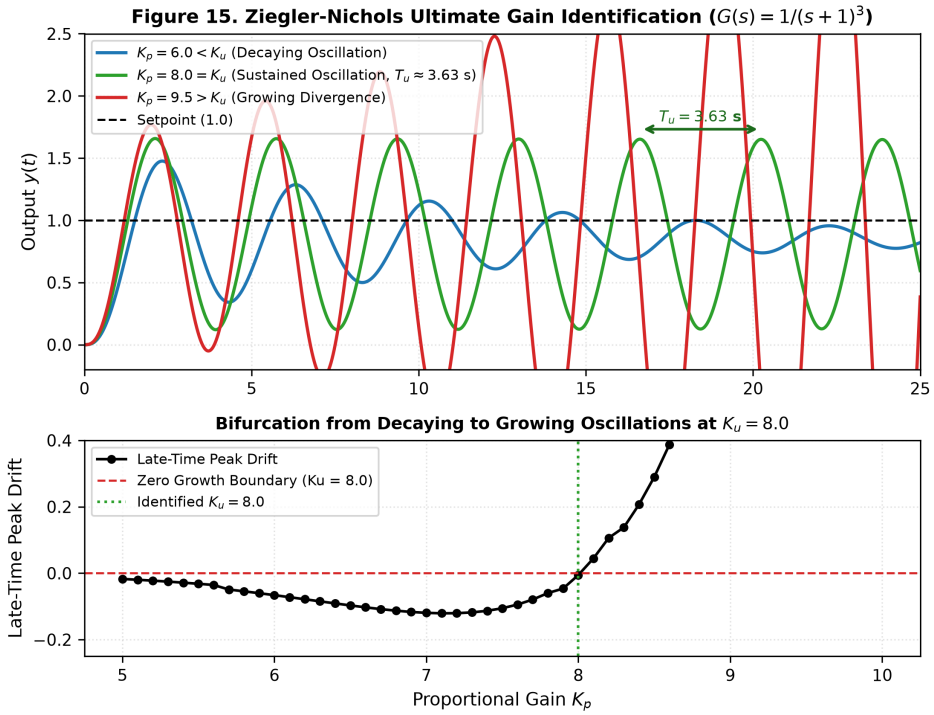


Figure 15: P-only control around K_u: decaying, sustained (period $T_u = 3.625\,\mathrm{s}$) and growing oscillation.

The classic PID rule is $K_p = 0.6K_u$, $T_i = T_u/2$, $T_d = T_u/8$, converted to parallel gains with $K_i = K_p/T_i$ and $K_d = K_pT_d$. It overshoots a lot, so I compared it with Tyreus–Luyben and searched for a better set (target: overshoot below 10% and settling below 10 s).

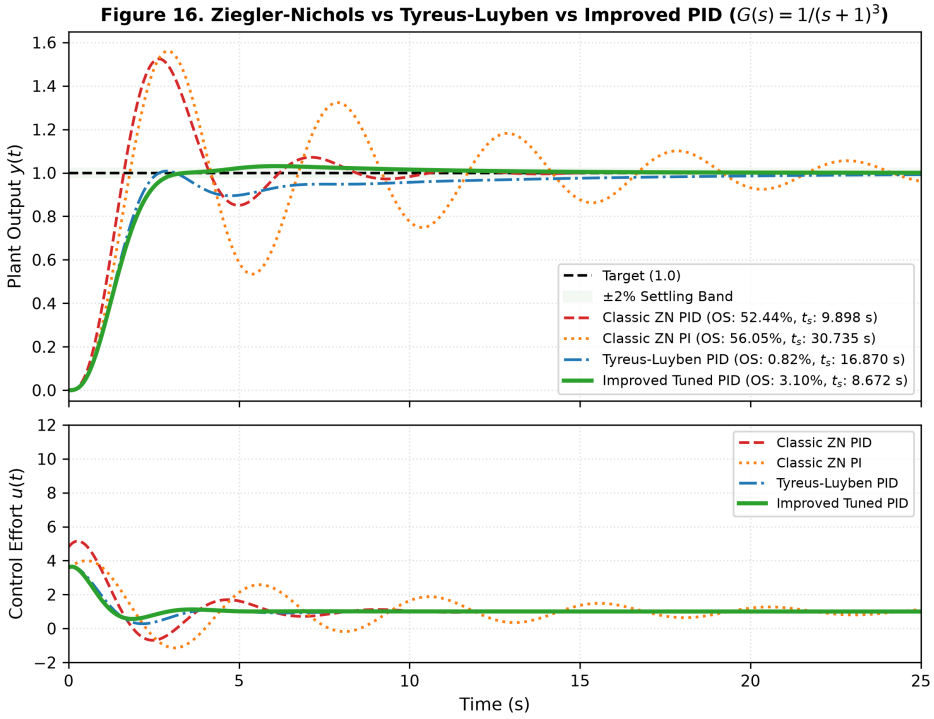


Figure 16: Classic Ziegler–Nichols, Tyreus–Luyben and my improved gains on $1/(s+1)^3$.

Method	$K_p/K_i/K_d$	overshoot	settling	rise
Classic ZN PID	4.80/2.65/2.18	52.4%	9.90 s	0.95 s
Classic ZN PI	3.60/1.19/0	56.1%	30.7 s	1.05 s
Tyreus–Luyben*	3.64/0.46/2.09	0.8%	16.9 s	1.51 s
Improved ($\tau = 0.02$ s)	**3.60/0.80/3.20**	**3.1%**	**8.67 s**	1.68 s

*with D filter $\tau = 0.02\,\mathrm{s}$ (0.96% overshoot unfiltered). Metrics measured over 35 s so that slow tails are included.

Takeaway. Ziegler–Nichols gives a starting point, not a finished design: here it needs retuning to cut 52% overshoot to 3% with settling under 9 s.

9 The code

The controller is one small class in `pidlab/pid.py` using only NumPy. Options (all switchable, so Section 7 can show before and after): output limits `u_min/u_max`; anti-windup `"clamp"`, `"none"` or `"backcalc"`; D on `"measurement"` or `"error"`; and a low-pass filter on D (`d_filter_tau`). Units: e in m, u in N, K_p in N/m, K_i in N/(m·s), K_d in N·s/m, times in s. Below is its `update()` method, called once per sample.

```python
def update(self, setpoint: float, measurement: float) -> float:
    """Calculate control force u given target setpoint and current measurement.

    Returns:
        u: Actuator force (clipped to [u_min, u_max]) in Newtons.
    """
    error = setpoint - measurement

    # 1. Proportional term
    p_term = self.kp * error

    # 2. Derivative term
    if self.d_on == "measurement":
        if self.prev_meas is None:
            d_raw = 0.0
        else:
            d_raw = -(measurement - self.prev_meas) / self.dt
    elif self.d_on == "error":
        if self.prev_error is None:
            d_raw = error / self.dt
        else:
            d_raw = (error - self.prev_error) / self.dt
    else:
        raise ValueError(f"Unknown d_on mode: '{self.d_on}'. Choose 'measurement' or 'error'.")

    # Derivative low-pass filter: d_f += alpha * (d_raw - d_f), alpha = dt / (tau + dt)
    if self.d_filter_tau > 0.0:
        alpha = self.dt / (self.d_filter_tau + self.dt)
        if self.prev_meas is None and self.prev_error is None:
            self.d_filtered = d_raw
        else:
            self.d_filtered += alpha * (d_raw - self.d_filtered)
        d_term = self.kd * self.d_filtered
    else:
        d_term = self.kd * d_raw

    # 3. Integral term & Anti-windup clamping
    candidate_integral = self.integral + error * self.dt
    candidate_i_term = self.ki * candidate_integral

    # Unconstrained candidate control output
    u_unsat = p_term + candidate_i_term + d_term

    if self.anti_windup == "clamp":
        # Clamping: stop accumulating if saturated and error pushes further into saturation
        saturating_high = (u_unsat > self.u_max) and (error > 0.0)
        saturating_low = (u_unsat < self.u_min) and (error < 0.0)

        if not (saturating_high or saturating_low):
            self.integral = candidate_integral
        i_term = self.ki * self.integral

    elif self.anti_windup == "none":
        self.integral = candidate_integral
        i_term = candidate_i_term

    elif self.anti_windup == "backcalc":
        u_clipped = float(np.clip(u_unsat, self.u_min, self.u_max))
        tt = self.kp if self.kp > 0 else 1.0
        self.integral += (error + (u_clipped - u_unsat) / tt) * self.dt
        i_term = self.ki * self.integral

    else:
        raise ValueError(f"Unknown anti_windup mode: '{self.anti_windup}'.")

    # Compute final saturated output
    u_final_unsat = p_term + i_term + d_term
    u_sat = float(np.clip(u_final_unsat, self.u_min, self.u_max))

    # Store state for next cycle and diagnostics
    self.prev_meas = measurement
```

```
        self.prev_error = error

        self.last_p = p_term
        self.last_i = i_term
        self.last_d = d_term
        self.last_u = u_sat
        self.last_u_unsat = u_final_unsat

        return u_sat
```

Run on the test system (30 N limit, all fixes on)

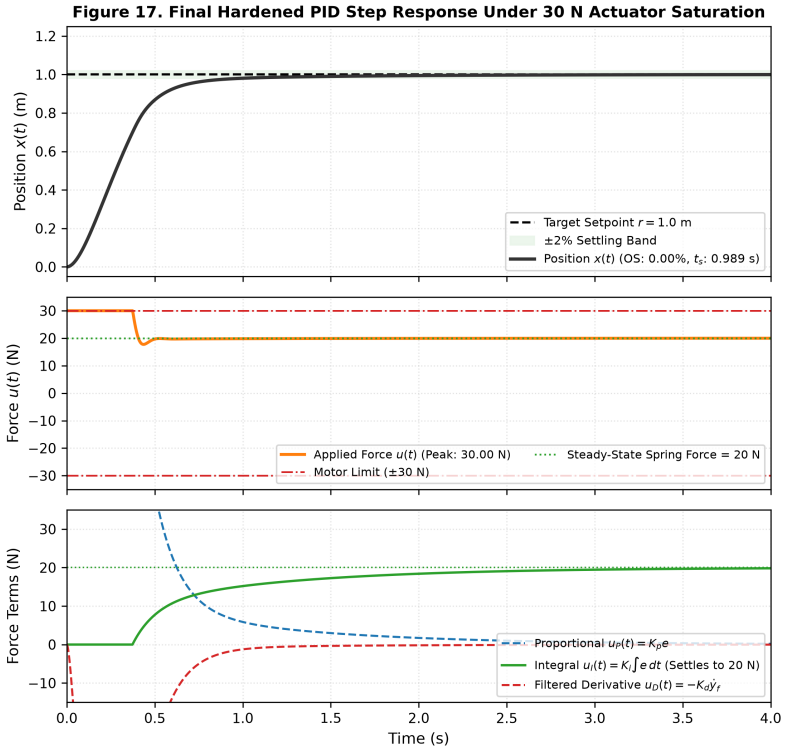


Figure 17: Final controller ($K_p = 300$, $K_i = 300$, $K_d = 30$, $\tau = 0.02\,\text{s}$, $|u| \le 30\,\text{N}$): position, force and the three terms.

```
Gains:  Kp=300 Ki=300 Kd=30 | clamp, D on measurement, tau=0.02 s | limit +/-30 N
rise 0.454 s | overshoot 0.00% | settling 0.989 s | steady error 0.0000 m | peak force 30.00 N
first 5 forces:  [30.0, 30.0, 30.0, 30.0, 30.0]
```

Compared with Section 6 (4.27% overshoot, rise 0.206 s), the 30 N limit makes the rise slower (0.454 s) and the clamping removes the overshoot. The force limit, not the gains, is what sets the speed.

10 Real robots and myths

10.1 Where PID is used

1. **Cruise control.** Error: target speed minus wheel speed. Actuator: throttle. Usually PI: integral action holds speed on hills; D is skipped because wheel-speed signals are noisy.
2. **Drone attitude.** Error: target angle minus IMU angle (outer loop), target rate minus gyro rate (inner loop). Actuator: motor speed controllers. D braking matters, and it must be filtered because motor vibration leaks into the gyro.
3. **Robot arm joint.** Error: target angle minus encoder angle. Actuator: motor torque. Holding an arm against gravity needs a steady torque, just like the 20 N spring force here, so PID is often paired with a gravity feedforward term.
4. **Line-following robot.** Error: sideways offset of the line from a sensor array. Actuator: difference in left/right wheel speed. Usually PD: P steers, D stops it zig-zagging.
5. **DC motor speed servo.** Error: target minus measured speed. Actuator: PWM voltage. Often a fast inner current loop inside a slower speed loop; I handles friction.

PID is among the most widely used controllers in industry; real systems pair it with feedforward, cascaded loops and filtering.

10.2 Five myths, tested

1. **"More gain is always better."** Raising K_p 100→1000 cuts steady error but overshoot goes 0%→57.6% and the starting force reaches 1000 N (Fig. 5).
2. **"I fixes everything."** With a 30 N limit the integral winds up: 17.6% overshoot and 2.98 s settling until clamping is added (Fig. 11).
3. **"D predicts the future."** D only reacts to the current slope, and it amplifies noise: 1 mm of sensor noise makes 43 N of force chatter (Fig. 12).
4. **"One set of gains fits all conditions."** The same 300/300/30 gains give 4.27% overshoot at 1 kg, 6.9% at 2 kg and 4.0% at 0.5 kg (run `playground.py -mass 2`).
5. **"Ziegler–Nichols gives the final gains."** It gave 52.4% overshoot here; retuning brought it to 3.1% (Fig. 16).

Reproducing the results, and what is missing

Reproduce: clone the repository, `pip install -r requirements.txt`, then `python run_all.py`, which regenerates all 17 figures and the tables in about 40 s. `python playground.py` lets you change gains, mass, noise or force limit and prints the four measures. A test script checks the main numbers against theory.
Method: semi-implicit Euler with $\Delta t = 1$ ms (halving Δt changes overshoot by 0.01 points); noise is Gaussian with fixed seed 42.
Limits of this work: everything is simulation of a linear plant; no hardware was used. The chosen gains are a good baseline, not the optimum (Section 6). The slow-update limit (about 60 ms) depends on the stated test (long-run growth).